

Язык программирования С

Лекция 2: встроенные типы, массивы, указатели

Лисица Никита

2026

Встроенные (базовые) типы данных

На большинстве современных платформ:

Тип	Размер в байтах	Диапазон значений
<code>char</code>	1	$-2^7 \dots 2^7 - 1$
<code>short</code>	2	$-2^{15} \dots 2^{15} - 1$
<code>int</code>	4	$-2^{31} \dots 2^{31} - 1$
<code>long</code>	4/8	$-2^{31/63} \dots 2^{31/63} - 1$
<code>long long</code>	8	$-2^{63} \dots 2^{63} - 1$
<code>unsigned char</code>	1	$0 \dots 2^8 - 1$
<code>unsigned short</code>	2	$0 \dots 2^{16} - 1$
<code>unsigned int</code>	4	$0 \dots 2^{32} - 1$
<code>unsigned long</code>	4/8	$0 \dots 2^{32/64} - 1$
<code>unsigned long long</code>	8	$0 \dots 2^{64} - 1$
<code>float</code>	4	$\pm 1.4 \times 10^{-45} \dots 3.4 \times 10^{38}$
<code>double</code>	8	$\pm 4.9 \times 10^{-324} \dots 1.8 \times 10^{308}$

Встроенные (базовые) типы данных

На большинстве современных платформ:

Тип	Размер в байтах	Диапазон значений
<code>char</code>	1	$-2^7 \dots 2^7 - 1$
<code>short</code>	2	$-2^{15} \dots 2^{15} - 1$
<code>int</code>	4	$-2^{31} \dots 2^{31} - 1$
<code>long</code>	4/8	$-2^{31/63} \dots 2^{31/63} - 1$
<code>long long</code>	8	$-2^{63} \dots 2^{63} - 1$
<code>unsigned char</code>	1	$0 \dots 2^8 - 1$
<code>unsigned short</code>	2	$0 \dots 2^{16} - 1$
<code>unsigned int</code>	4	$0 \dots 2^{32} - 1$
<code>unsigned long</code>	4/8	$0 \dots 2^{32/64} - 1$
<code>unsigned long long</code>	8	$0 \dots 2^{64} - 1$
<code>float</code>	4	$\pm 1.4 \times 10^{-45} \dots 3.4 \times 10^{38}$
<code>double</code>	8	$\pm 4.9 \times 10^{-324} \dots 1.8 \times 10^{308}$

• Почему ‘на большинстве’?

Встроенные (базовые) типы данных

- Стандарт языка C гарантирует только **минимальный размер** встроенных типов и неравенства на их порядок

Встроенные (базовые) типы данных

- Стандарт языка C гарантирует только **минимальный размер** встроенных типов и неравенства на их порядок
- **char** – всегда 1 байт (почти всегда 8 бит)

Встроенные (базовые) типы данных

- Стандарт языка C гарантирует только **минимальный размер** встроенных типов и неравенства на их порядок
- **char** – всегда 1 байт (почти всегда 8 бит)
- **int** – естественный для CPU тип ($\approx$ размер регистров), как минимум 16 бит (обычно 32)

Встроенные (базовые) типы данных

- Стандарт языка C гарантирует только **минимальный размер** встроенных типов и неравенства на их порядок
- **char** – всегда 1 байт (почти всегда 8 бит)
- **int** – естественный для CPU тип ($\approx$ размер регистров), как минимум 16 бит (обычно 32)
- **sizeof(short) $\leq$ sizeof(int) $\leq$ sizeof(long)**

Встроенные (базовые) типы данных

- Стандарт языка C гарантирует только **минимальный размер** встроенных типов и неравенства на их порядок
- **char** – всегда 1 байт (почти всегда 8 бит)
- **int** – естественный для CPU тип ($\approx$ размер регистров), как минимум 16 бит (обычно 32)
- **sizeof(short) $\leq$ sizeof(int) $\leq$ sizeof(long)**
- **sizeof** – встроенный в язык оператор (не функция!), в момент компиляции его вызов заменяется на размер типа или значения

- Представление отрицательных чисел в дополнительном коде

Изучить (вспомнить) самостоятельно

- Представление отрицательных чисел в дополнительном коде
- Представление чисел с плавающей точкой

Изучить (вспомнить) самостоятельно

- Представление отрицательных чисел в дополнительном коде
- Представление чисел с плавающей точкой
- Приведение типов (явное, неявное)

Изучить (вспомнить) самостоятельно

- Представление отрицательных чисел в дополнительном коде
- Представление чисел с плавающей точкой
- Приведение типов (явное, неявное)
- Что-то из этого будет в письменном тесте :)

- Последовательность объектов одного типа, лежащих в непрерывном участке памяти
- Размер должен быть известен **на момент компиляции!**

```
// Массив из 10-ти элементов типа int
// Размер в памяти: 10 * sizeof(int)
int array[10];

// Инициализация:
int array[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

// Заполнить нулями:
int array[10] = {0};

// Вывести размер из инициализации:
int array[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
char array[10];  
  
// Прочитать/записать значение по индексу:  
array[0] = 10;  
array[1] = array[0] + 5;  
  
// Узнать размер:  
int size = sizeof(array) / sizeof(array[0]);
```

Массивы char

```
// Массив из 10-ти элементов типа char
// Размер в памяти: 10 * sizeof(int)
char array[10];

char array[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

char array[] = {'H', 'e', 'l', 'l', 'o', ',', ' ',
               ' ', 'w', 'o', 'r', 'l', 'd', '!', '\0'};

// То же самое через строковой литерал
// 14 элементов - включает терминирующий '\0'
char array[] = "Hello, world!";

// NB: вот такой синтаксис - абсолютно другая операция!
char *array = "Hello, world!";
```

Многомерные массивы

```
// Массив из 3-х элементов, каждый - массив из 4-х int-ов
// Размер в памяти: 3 * 4 * sizeof(int)
int array[3][4];

int array[2][2] = {
    {1, 2},
    {3, 4},
};
```

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – `undefined behavior`

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – **undefined behavior**
- Что произойдёт на практике, если написать `array[-1]`?

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – **undefined behavior**
- Что произойдёт на практике, если написать `array[-1]`?
- На современных компиляторах будет предупреждение/ошибка

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – **undefined behavior**
- Что произойдёт на практике, если написать `array[-1]`?
- На современных компиляторах будет предупреждение/ошибка
- Если индекс неизвестен в момент компиляции (переменная/аргумент функции/т.п.), процессор попытается прочесть память за пределами массива

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – **undefined behavior**
- Что произойдёт на практике, если написать `array[-1]`?
- На современных компиляторах будет предупреждение/ошибка
- Если индекс неизвестен в момент компиляции (переменная/аргумент функции/т.п.), процессор попытается прочесть память за пределами массива
- Если это память того же процесса – ошибки не будет, но будет прочитано 'мусорное' значение

Выход за пределы массива

- Для массива `int array[N];` обращение по индексу $i < 0$ или $i \geq N$ – **undefined behavior**
- Что произойдёт на практике, если написать `array[-1]`?
- На современных компиляторах будет предупреждение/ошибка
- Если индекс неизвестен в момент компиляции (переменная/аргумент функции/т.п.), процессор попытается прочесть память за пределами массива
- Если это память того же процесса – ошибки не будет, но будет прочитано ‘мусорное’ значение
- Если это память другого процесса – access violation / segmentation fault

- Указатель (pointer) – объект, указывающий/ссылающийся на другой объект

- Указатель (pointer) – объект, указывающий/ссылающийся на другой объект
- На практике на большинстве платформ указатель – адрес в памяти (число), номер первого байта объекта

- Указатель (pointer) – объект, указывающий/ссылающийся на другой объект
- На практике на большинстве платформ указатель – адрес в памяти (число), номер первого байта объекта
- **N.B.:** это адрес не в *физической памяти*, а в *виртуальной*, но это не принципиально для понимания

- Указатель (pointer) – объект, указывающий/ссылающийся на другой объект
- На практике на большинстве платформ указатель – адрес в памяти (число), номер первого байта объекта
- **N.B.:** это адрес не в *физической памяти*, а в *виртуальной*, но это не принципиально для понимания
- `int *p;` – указатель на кусок памяти, в котором хранится число типа `int`

```
int a = 42;

// Взятие адреса переменной:
int *p = &a;

// "Разыменование" указателя
// Прочитать значение по адресу
int b = *p;

// "Разыменование" указателя
// Записать значение по адресу
*p = 67;

// Напечатать указатель
printf("%p", p);
```

- Стандарт C не регламентирует размер самого указателя, но он одинаков для всех типов указателей (кроме указателей на функции)

- Стандарт C не регламентирует размер самого указателя, но он одинаков для всех типов указателей (кроме указателей на функции)
- На практике размер указателя зависит от битности системы: 4 байта на 32-битной (x86), 8 байт на 64-битной (x86_64, ARM64)

Размер указателя

- Не путайте размер указателя и размер объекта, на который ссылается указатель!

```
char c = 'A';  
int i = 10;  
double d = 3.1415;
```

```
// На x86\_64  
sizeof(c) == 1;  
sizeof(i) == 4;  
sizeof(d) == 8;  
sizeof(&c) == 8;  
sizeof(&i) == 8;  
sizeof(&d) == 8;
```

- К указателям можно прибавлять/вычитать число

- К указателям можно прибавлять/вычитать число
- Семантика такой операции – перейти к следующему элементу того же типа в непрерывном участке памяти/массиве

- К указателям можно прибавлять/вычитать число
- Семантика такой операции – перейти к следующему элементу того же типа в непрерывном участке памяти/массиве
- $\implies$ К адресу, хранящемуся в указателе, прибавляется число, умноженное на `sizeof(*p)`

Арифметика указателей

```
char array_c[] = "MKN rules";
int array_i[10];
double array_d[10];

char *pc = &array_c[0];
int *pi = &array_i[0];
double *pd = &array_d[0];

// К фактическому адресу прибавится sizeof(char) == 1 байт
pc += 1;

// К фактическому адресу прибавится sizeof(int) == 4 байт
pi += 1;

// К фактическому адресу прибавится sizeof(double) == 8 байт
pd += 1;
```

- Аналогично, вычитание указателей даёт количество элементов между ними (а не байт!)

```
int array[10];  
  
int *p1 = &array[3];  
int *p2 = &array[7];  
  
// delta == 4  
int delta = p2 - p1;
```

- Массив автоматически приводится к указателю на первый элемент массива:

```
int array[10];  
// То же самое, что p = &array[0]  
int *p = array;  
  
// Ошибка компиляции: массиву нельзя присвоить новый адрес  
array = p;
```

Указатели и массивы

- Массив автоматически приводится к указателю на первый элемент массива:

```
int array[10];  
// То же самое, что p = &array[0]  
int *p = array;  
  
// Ошибка компиляции: массиву нельзя присвоить новый адрес  
array = p;
```

- Через указатель тоже можно обращаться по индексу;

```
int *p = ...;  
  
p[2] = p[1] + p[0];
```

Указатели и массивы

- Массив автоматически приводится к указателю на первый элемент массива:

```
int array[10];  
// То же самое, что p = &array[0]  
int *p = array;  
  
// Ошибка компиляции: массиву нельзя присвоить новый адрес  
array = p;
```

- Через указатель тоже можно обращаться по индексу;

```
int *p = ...;  
  
p[2] = p[1] + p[0];
```

- `p[i]` эквивалентно `*(p + i)`

- `p[i]` эквивалентно `*(p + i)`

- `p[i]` эквивалентно `*(p + i)`
- Сложение коммутативно: `*(p + i) == *(i + p)`

- `p[i]` эквивалентно `*(p + i)`
- Сложение коммутативно: `*(p + i) == *(i + p)`
- $\implies$ Такой синтаксис тоже работает: `i[p]`

- `p[i]` эквивалентно `*(p + i)`
- Сложение коммутативно: `*(p + i) == *(i + p)`
- $\implies$ Такой синтаксис тоже работает: `i[p]`
- Не стоит так делать в реальном коде :)

Приведение типов указателей

- В языке C любые типы указателей приводятся друг к другу:

```
int  *pi = ...;  
char *pc = ...;  
  
// ОК в C, ошибка в C++  
pi = pc;
```

Приведение типов указателей

- В языке C любые типы указателей приводятся друг к другу:

```
int  *pi = ...;  
char *pc = ...;  
  
// ОК в C, ошибка в C++  
pi = pc;
```

- В C++ потребуется явное приведение типа:

```
pi = (int*) pc;
```

Применение указателей

- Хотим написать функцию, меняющую местами два значения

```
void swap(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int x = 3, y = 4;  
    swap(x, y);  
    return 0;  
}
```

Применение указателей

- Хотим написать функцию, меняющую местами два значения

```
void swap(int a, int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
int main() {  
    int x = 3, y = 4;  
    swap(x, y);  
    return 0;  
}
```

- Функция `swap` поменяет местами **копии** аргументов, а не сами переменные `x, y`

Применение указателей

- Решение:

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 3, y = 4;  
    swap(&x, &y);  
    return 0;  
}
```

Применение указателей

- Решение:

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 3, y = 4;  
    swap(&x, &y);  
    return 0;  
}
```

- В функцию `swap` передаются **копии адресов** переменных `x, y`
⇒ функция может поменять значения самих переменных

Применение указателей

- Хотим передать в функцию большой объект и избежать лишнего копирования:

```
big_type big_object = ...;  
foo(&big_object);
```

Применение указателей

- Хотим передать в функцию большой объект и избежать лишнего копирования:

```
big_type big_object = ...;  
foo(&big_object);
```

```
char str[] = "Lorem ipsum dolor sit amet, ...";  
int size = strlen(str);
```

Применение указателей

- Хотим пройтись по большому массиву:

```
int strlen (char *ptr) {  
    int len = 0;  
    while (ptr[len] != '\0') {  
        ++len;  
    }  
    return len;  
}
```

```
int strlen (char *ptr) {  
    char *p = ptr;  
    while (*p != '\0') {  
        ++p;  
    }  
    return p - ptr;  
}
```

Применение указателей

- Хотим пройтись по большому массиву:

```
int strlen (char *ptr) {  
    int len = 0;  
    while (ptr[len] != '\0') {  
        ++len;  
    }  
    return len;  
}
```

```
int strlen (char *ptr) {  
    char *p = ptr;  
    while (*p != '\0') {  
        ++p;  
    }  
    return p - ptr;  
}
```

- `ptr[len]` раскрывается в `*(ptr + len)` $\implies$ лишнее сложение в сравнении с правым вариантом